Plan

Plan

Four (perhaps five) lectures on the MatterMost paper:

- The theory behind the MatterMost paper (three lectures, perhaps four)
- The main idea of the MatterMost paper (one lecture)

Plan

Four (perhaps five) lectures on the MatterMost paper:

- The theory behind the MatterMost paper (three lectures, perhaps four)
- The main idea of the MatterMost paper (one lecture)

Today:

- The theory behind the MatterMost paper 3

Wrap up MM2 lecture

Wrap up MM2 lecture

- We have started to look into the theory behind the MM paper.

Wrap up MM2 lecture

- We have started to look into the theory behind the MM paper.
- We have seen that this consists of software components for specifying and for solving finite horizon SDPs.

Wrap up MM2 lecture

- We have started to look into the theory behind the MM paper.
- We have seen that this consists of software components for specifying and for solving finite horizon SDPs.
- We have discussed most of the specification components and ...

Wrap up MM2 lecture

- We have started to look into the theory behind the MM paper.
- We have seen that this consists of software components for specifying and for solving finite horizon SDPs.
- We have discussed most of the specification components and ...
- ...we have started to specify what it means to solve a SDP.

Wrap up MM2 lecture

Specifically, we have introduced:

$$
\begin{array}{lcl}
M & : & Set \to Set \\
fmap_M & : & \{A\ B : Set\} \to (A \to B) \to M\ A \to M\ B \\
\eta_M & : & \{A : Set\} \to A \to M\ A \\
\mu_M & : & \{A : Set\} \to M\ (M\ A) \to M\ A
\end{array}
$$

Wrap up MM2 lecture

Specifically, we have introduced:

$$
\begin{aligned}
M &: Set \to Set \\
fmap_M &: \{A\ B : Set\} \to (A \to B) \to M\ A \to M\ B \\
\eta_M &: \{A : Set\} \to A \to M\ A \\
\mu_M &: \{A : Set\} \to M\ (M\ A) \to M\ A
\end{aligned}
$$

$$
\begin{aligned}
&_\gg\!=_M_ : \{B\ C : Set\} \to M\ B \to (B \to M\ C) \to M\ C \\
&mb \gg\!=_M f = \mu_M\ (fmap_M\ f\ mb)
\end{aligned}
$$

$$
\begin{aligned}
&_>\!=\!>_M_ : \{A\ B\ C : Set\} \to (A \to M\ B) \to (B \to M\ C) \to (A \to M\ C) \\
&(f >\!=\!>_M g)\ a = (f\ a) \gg\!=_M g
\end{aligned}
$$

Wrap up MM2 lecture

$$
\begin{aligned}
X &\ : \mathbb{N} \to \mathit{Set} \\
Y &\ : (t : \mathbb{N}) \to X\ t \to \mathit{Set} \\
\mathit{next} &\ : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to M\ (X\ (\mathit{suc}\ t))
\end{aligned}
$$

Wrap up MM2 lecture

$$
\begin{aligned}
X &\ : \mathbb{N} \to \mathit{Set} \\
Y &\ : (t : \mathbb{N}) \to X\ t \to \mathit{Set} \\
\mathit{next} &\ : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to M\ (X\ (\mathit{suc}\ t))
\end{aligned}
$$

$$\mathit{Val} : \mathit{Set}$$

$$
\begin{aligned}
\leqslant &\ : \mathit{Val} \to \mathit{Val} \to \mathit{Set} \\
\mathit{refl}_{\leqslant} &\ : (x : \mathit{Val}) \to x \leqslant x \\
\mathit{trans}_{\leqslant} &\ : (x\ y\ z : \mathit{Val}) \to x \leqslant y \to y \leqslant z \to x \leqslant z \\
\mathit{total}_{\leqslant} &\ : (x\ y : \mathit{Val}) \to \mathit{Either}\ (x \leqslant y)\ (y \leqslant x)
\end{aligned}
$$

Wrap up MM2 lecture

$$
\begin{aligned}
X &: \mathbb{N} \to Set \\
Y &: (t : \mathbb{N}) \to X\ t \to Set \\
next &: (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to M\ (X\ (suc\ t))
\end{aligned}
$$

$$
Val : Set
$$

$$
\begin{aligned}
\leqslant &: Val \to Val \to Set \\
refl_{\leqslant} &: (x : Val) \to x \leqslant x \\
trans_{\leqslant} &: (x\ y\ z : Val) \to x \leqslant y \to y \leqslant z \to x \leqslant z \\
total_{\leqslant} &: (x\ y : Val) \to Either\ (x \leqslant y)\ (y \leqslant x)
\end{aligned}
$$

$$
\begin{aligned}
0_{Val} &: Val \\
\oplus &: Val \to Val \to Val
\end{aligned}
$$

Wrap up MM2 lecture

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

Wrap up MM2 lecture

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.

Wrap up MM2 lecture

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.

$$\mathit{Policy} : (t : \mathbb{N}) \to \mathit{Set}$$
$$\mathit{Policy}\ t = (x : X\ t) \to Y\ t\ x$$

$$\textbf{data}\ \mathit{PolicySeq} : (t\ n : \mathbb{N}) \to \mathit{Set}\ \textbf{where}$$
$$\mathit{Nil} : \{t : \mathbb{N}\} \to \mathit{PolicySeq}\ t\ \mathit{zero}$$
$$_::_ : \{t\ n : \mathbb{N}\} \to \mathit{Policy}\ t \to \mathit{PolicySeq}\ (\mathit{suc}\ t)\ n \to \mathit{PolicySeq}\ t\ (\mathit{suc}\ n)$$

Example: crossing a non-deterministic river

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.
- Crossing takes 5 steps. Ideally, we want to reach the other side at the same position but the river will tend to drift the boat downstream towards the waterfall ...

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.
- Crossing takes 5 steps. Ideally, we want to reach the other side at the same position but the river will tend to drift the boat downstream towards the waterfall ...
- ... unless we are in the waterfall, at each step we can steer the boat upstream, downstream or just ahead.

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.
- Crossing takes 5 steps. Ideally, we want to reach the other side at the same position but the river will tend to drift the boat downstream towards the waterfall ...
- ... unless we are in the waterfall, at each step we can steer the boat upstream, downstream or just ahead.
- As we move from the shore towards the middle of the river, the flow becomes more turbulent and thus less predictable. Perhaps, in the middle of the river, the flow at certain positions even turns upstream because of whirls.

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.
- Crossing takes 5 steps. Ideally, we want to reach the other side at the same position but the river will tend to drift the boat downstream towards the waterfall ...
- ... unless we are in the waterfall, at each step we can steer the boat upstream, downstream or just ahead.
- As we move from the shore towards the middle of the river, the flow becomes more turbulent and thus less predictable. Perhaps, in the middle of the river, the flow at certain positions even turns upstream because of whirls.
- We seek a sequence of policies that avoids the waterfall and reaches the other side as near as possible to the starting position.

Example: crossing a non-deterministic river

- We want to find an optimal policy sequence for crossing a non-deterministic river in a boat starting from a safe position far upstream a waterfall.
- Crossing takes 5 steps. Ideally, we want to reach the other side at the same position but the river will tend to drift the boat downstream towards the waterfall ...
- ... unless we are in the waterfall, at each step we can steer the boat upstream, downstream or just ahead.
- As we move from the shore towards the middle of the river, the flow becomes more turbulent and thus less predictable. Perhaps, in the middle of the river, the flow at certain positions even turns upstream because of whirls.
- We seek a sequence of policies that avoids the waterfall and reaches the other side as near as possible to the starting position.
- We also want to minimize moving against the stream because this costs a lot of energy.

Example: crossing a non-deterministic river

- Let's specify the problem as a SDP! The river is non-deterministic:

$$
\begin{aligned}
M &= List \\
fmap_M &= fmap_{List} \\
\eta_M &= \eta_{List} \\
\mu_M &= \mu_{List}
\end{aligned}
$$

Example: crossing a non-deterministic river

- Let's specify the problem as a SDP! The river is non-deterministic:

$$
\begin{aligned}
M &= List \\
fmap_M &= fmap_{List} \\
\eta_M &= \eta_{List} \\
\mu_M &= \mu_{List}
\end{aligned}
$$

- It is also very long upstream the watefall:

$$X\ t = \mathbb{N}$$

Example: crossing a non-deterministic river

- Let's specify the problem as a SDP! The river is non-deterministic:

$$
\begin{aligned}
M &= List \\
fmap_M &= fmap_{List} \\
\eta_M &= \eta_{List} \\
\mu_M &= \mu_{List}
\end{aligned}
$$

- It is also very long upstream the watefall:

$$X\ t = \mathbb{N}$$

- If we end up in the waterfall, we have no control options ...

$$Y\ t\ zero = \top$$

Example: crossing a non-deterministic river

- Let's specify the problem as a SDP! The river is non-deterministic:

$$
\begin{aligned}
M &= List \\
fmap_M &= fmap_{List} \\
\eta_M &= \eta_{List} \\
\mu_M &= \mu_{List}
\end{aligned}
$$

- It is also very long upstream the watefall:

$$X\ t = \mathbb{N}$$

- If we end up in the waterfall, we have no control options ...

$$Y\ t\ zero = \top$$

- ...otherwise, we can steer Downstream, Upstream or Ahead:

$$Y\ t\ (suc\ x) = DUA$$

Example: crossing a non-deterministic river

- If we end up in the waterfall, we get stuck there:

$$next\ _\ zero\ _\ =\ zero\ ::\ []$$

Example: crossing a non-deterministic river

- If we end up in the waterfall, we get stuck there:

$$\mathit{next}\ _\ \mathit{zero}\ _\ =\ \mathit{zero}\ ::\ []$$

- Otherwise we move to a new position depending on the control we pick up and on the non-deterministic speed of the water, for example:

$$\begin{array}{lll}
\mathit{next}\ t\ \ \mathit{zero}\ \ _ & = & \mathit{zero}\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ D & = & \mathit{zero}\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ U & = & (\mathit{suc}\ \mathit{zero})\ ::\ (\mathit{suc}\ (\mathit{suc}\ \mathit{zero}))\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ A & = & \mathit{zero}\ ::\ (\mathit{suc}\ \mathit{zero})\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ D & = & x\ ::\ \mathit{suc}\ x\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ U & = & \mathit{suc}\ (\mathit{suc}\ x)\ ::\ \mathit{suc}\ (\mathit{suc}\ (\mathit{suc}\ x))\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ A & = & \mathit{suc}\ x\ ::\ \mathit{suc}\ (\mathit{suc}\ x)\ ::\ []
\end{array}$$

Example: crossing a non-deterministic river

- If we end up in the waterfall, we get stuck there:

$$\mathit{next}\ _\ \mathit{zero}\ _\ =\ \mathit{zero}\ ::\ []$$

- Otherwise we move to a new position depending on the control we pick up and on the non-deterministic speed of the water, for example:

$$\begin{array}{lcl}
\mathit{next}\ t\ \ \mathit{zero}\ \ _ & = & \mathit{zero}\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ D & = & \mathit{zero}\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ U & = & (\mathit{suc}\ \mathit{zero})\ ::\ (\mathit{suc}\ (\mathit{suc}\ \mathit{zero}))\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ \mathit{zero})\ \ A & = & \mathit{zero}\ ::\ (\mathit{suc}\ \mathit{zero})\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ D & = & x\ ::\ \mathit{suc}\ x\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ U & = & \mathit{suc}\ (\mathit{suc}\ x)\ ::\ \mathit{suc}\ (\mathit{suc}\ (\mathit{suc}\ x))\ ::\ [] \\
\mathit{next}\ t\ (\mathit{suc}\ (\mathit{suc}\ x))\ A & = & \mathit{suc}\ x\ ::\ \mathit{suc}\ (\mathit{suc}\ x)\ ::\ []
\end{array}$$

- We can account for the fact that the flow in the middle of the river is more turbulent by making *next* explicitely depend on t.

Example: crossing a non-deterministic river

- We still have to define *Val* and the reward function:

$$
\begin{aligned}
Val &= \mathbb{R} \\
0_{Val} &= 0.0 \\
\oplus &= _+_{\mathbb{R}}_
\end{aligned}
$$

Example: crossing a non-deterministic river

- We still have to define *Val* and the reward function:

$$
\begin{aligned}
Val &= \mathbb{R} \\
0_{Val} &= 0.0 \\
\oplus &= _+_{\mathbb{R}}_
\end{aligned}
$$

- We want to avoid the waterfall, thus we assign a low reward to any transition to position *zero*:

$$reward\ t\ x\ y\ zero = -1.0$$

Example: crossing a non-deterministic river

- We still have to define *Val* and the reward function:

$$
\begin{aligned}
Val &= \mathbb{R} \\
0_{Val} &= 0.0 \\
\oplus &= _+_{\mathbb{R}}_
\end{aligned}
$$

- We want to avoid the waterfall, thus we assign a low reward to any transition to position *zero*:

$$reward\ t\ x\ y\ zero\ =\ -1.0$$

- How do we account for the goals of reaching the other side as near as possible to the starting position and to minimize the costs of moving against the stream?

$$reward\ t\ x\ y\ (suc\ x')\ =\ ?$$

Back to the theory!

Solution: state-control sequences

Solution: state-control sequences

- ... a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.

Solution: state-control sequences

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Remember that *next* represents a decision network. With an initial state and a policy sequence, it defines a unique tree of possible state-control sequences:

Solution: state-control sequences

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Remember that *next* represents a decision network. With an initial state and a policy sequence, it defines a unique tree of possible state-control sequences:

$$
\begin{array}{l}
\textbf{data}\ \mathit{XYSeq}\ :\ (t\ n\ :\ \mathbb{N})\ \rightarrow\ \mathit{Set}\ \textbf{where} \\
\quad \mathit{Last}\ :\ \{t\ :\ \mathbb{N}\}\ \ \rightarrow\ X\ t\ \rightarrow\ \mathit{XYSeq}\ t\ (\mathit{suc}\ \mathit{zero}) \\
\quad _{+\!+}_\ \ :\ \{t\ n\ :\ \mathbb{N}\}\ \rightarrow\ \Sigma\ (X\ t)\ (Y\ t)\ \rightarrow\ \mathit{XYSeq}\ (\mathit{suc}\ t)\ (\mathit{suc}\ n)\ \rightarrow\ \mathit{XYSeq}\ t\ (\mathit{suc}\ (\mathit{suc}\ n))
\end{array}
$$

Solution: state-control sequences

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Remember that *next* represents a decision network. With an initial state and a policy sequence, it defines a unique tree of possible state-control sequences:

$$
\begin{array}{l}
\textbf{data}\ \mathit{XYSeq}\ :\ (t\ n\ :\ \mathbb{N})\ \rightarrow\ \mathit{Set}\ \textbf{where} \\
\quad \mathit{Last}\ :\ \{t\ :\ \mathbb{N}\}\ \rightarrow\ X\ t\ \rightarrow\ \mathit{XYSeq}\ t\ (\mathit{suc}\ \mathit{zero}) \\
\quad _{+\!+}_\ :\ \{t\ n\ :\ \mathbb{N}\}\ \rightarrow\ \Sigma\ (X\ t)\ (Y\ t)\ \rightarrow\ \mathit{XYSeq}\ (\mathit{suc}\ t)\ (\mathit{suc}\ n)\ \rightarrow\ \mathit{XYSeq}\ t\ (\mathit{suc}\ (\mathit{suc}\ n))
\end{array}
$$

Exercise 6.1

A value of type *XYSeq t n* is like a vector. What is its length? Can *n* be zero? Why is the first constructor of *XYSeq* called *Last*?

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- The possible trajectories of a policy sequence are computed by induction (remember OUU3!):

$$
\begin{aligned}
&\mathit{trj} : \{t\ n : \mathbb{N}\} \to \mathit{PolicySeq}\ t\ n \to X\ t \to M\ (\mathit{XYSeq}\ t\ (\mathit{suc}\ n)) \\
&\mathit{trj}\ \{t\}\ \mathit{Nil}\qquad x = \eta_M\ (\mathit{Last}\ x) \\
&\mathit{trj}\ \{t\}\ (p :: \mathit{ps})\ x = \mathbf{let}\ y \quad = p\ x\ \mathbf{in} \\
&\qquad\qquad\qquad\qquad\quad \mathbf{let}\ \mathit{mx}' = \mathit{next}\ t\ x\ y\ \mathbf{in} \\
&\qquad\qquad\qquad\qquad\quad \mathit{fmap}_M\ ((x\ ,\ y) \shortparallel_)\ (\mathit{mx}' \gg\!=_M \mathit{trj}\ \mathit{ps})
\end{aligned}
$$

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- The possible trajectories of a policy sequence are computed by induction (remember OUU3!):

$$
\begin{aligned}
&\mathit{trj} : \{t\ n : \mathbb{N}\} \to \mathit{PolicySeq}\ t\ n \to X\ t \to M\ (\mathit{XYSeq}\ t\ (\mathit{suc}\ n)) \\
&\mathit{trj}\ \{t\}\ \mathit{Nil}\qquad x = \eta_M\ (\mathit{Last}\ x) \\
&\mathit{trj}\ \{t\}\ (p :: \mathit{ps})\ x = \mathbf{let}\ y \quad = p\ x\ \mathbf{in} \\
&\qquad\qquad\qquad\qquad\ \ \mathbf{let}\ \mathit{mx}' = \mathit{next}\ t\ x\ y\ \mathbf{in} \\
&\qquad\qquad\qquad\qquad\ \ \mathit{fmap}_M\ ((x\ ,\ y) \shortparallel_)\ (\mathit{mx}' \gg\!=_M \mathit{trj}\ \mathit{ps})
\end{aligned}
$$

Exercise 6.2

Make sure that you understand the computation of possible trajectories. What are the types of *Nil*, $(p :: \mathit{ps})$ and x? What is the type of $\eta_M\ (\mathit{Last}\ x)$? What are the types of y, mx' in the **let-in** clauses?

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Now we can compute the $\oplus$-sum of the rewards along all possible trajectories ...

$$
\begin{aligned}
&\mathit{sumR} : \{t\ n : \mathbb{N}\} \to \mathit{XYSeq}\ t\ n \to \mathit{Val} \\
&\mathit{sumR}\ \{t\}\ (\mathit{Last}\ x) = 0_{\mathit{Val}} \\
&\mathit{sumR}\ \{t\}\ ((x\ ,\ y) \mathbin{++} \mathit{xys}) = \mathit{reward}\ t\ x\ y\ (\mathit{head}\ \mathit{xys}) \oplus \mathit{sumR}\ \mathit{xys}
\end{aligned}
$$

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Now we can compute the $\oplus$-sum of the rewards along all possible trajectories ...

$$
\begin{aligned}
&\mathit{sumR} : \{t\ n : \mathbb{N}\} \to \mathit{XYSeq}\ t\ n \to \mathit{Val} \\
&\mathit{sumR}\ \{t\}\ (\mathit{Last}\ x) \qquad\qquad = 0_{\mathit{Val}} \\
&\mathit{sumR}\ \{t\}\ ((x\ ,\ y) \mathbin{++} \mathit{xys}) = \mathit{reward}\ t\ x\ y\ (\mathit{head}\ \mathit{xys}) \oplus \mathit{sumR}\ \mathit{xys}
\end{aligned}
$$

- ...and the value of taking n decisions according to a policy sequence in an initial state:

$$
\begin{aligned}
&\mathit{val} : \{t\ n : \mathbb{N}\} \to (\mathit{ps} : \mathit{PolicySeq}\ t\ n) \to (x : X\ t) \to \mathit{Val} \\
&\mathit{val}\ \mathit{ps} = \mathit{measure} \circ \mathit{fmap}_M\ \mathit{sumR} \circ \mathit{trj}\ \mathit{ps}
\end{aligned}
$$

Solution: possible trajectories

- ...a pol. seq. that max. a meas. of the $\oplus$-sum of the rewards along all possible trajectories.
- Now we can compute the $\oplus$-sum of the rewards along all possible trajectories ...

$$
\begin{aligned}
&\mathit{sumR} : \{t\ n : \mathbb{N}\} \to \mathit{XYSeq}\ t\ n \to \mathit{Val} \\
&\mathit{sumR}\ \{t\}\ (\mathit{Last}\ x) \qquad\qquad = 0_{\mathit{Val}} \\
&\mathit{sumR}\ \{t\}\ ((x\ ,\ y) \mathbin{++} \mathit{xys}) = \mathit{reward}\ t\ x\ y\ (\mathit{head}\ \mathit{xys}) \oplus \mathit{sumR}\ \mathit{xys}
\end{aligned}
$$

- ...and the value of taking n decisions according to a policy sequence in an initial state:

$$
\begin{aligned}
&\mathit{val} : \{t\ n : \mathbb{N}\} \to (\mathit{ps} : \mathit{PolicySeq}\ t\ n) \to (x : X\ t) \to \mathit{Val} \\
&\mathit{val}\ \mathit{ps} = \mathit{measure} \circ \mathit{fmap}_M\ \mathit{sumR} \circ \mathit{trj}\ \mathit{ps}
\end{aligned}
$$

Exercise 6.3

What are the types of *head* and *measure* in the definitions of *sumR* and *val*? Define *head*. A popular *measure* for $\mathit{Val} = \mathbb{R}$ and $M\ A = \mathit{SP}\ A = \mathit{List}\ (A \times \mathbb{R}_{[0,1]})$ (finite probability distributions) is the expected value measure. Define this measure.

Solution: optimality of policy sequences

- A solution of a SDP is a policy sequence that maximizes the measure of the $\oplus$-sum of the rewards along all possible trajectories.

Solution: optimality of policy sequences

- A solution of a SDP is a policy sequence that maximizes the measure of the $\oplus$-sum of the rewards along all possible trajectories.
- Now we can express what it means for a policy sequence to be a solution of a SDP precisely:

Solution: optimality of policy sequences

- A solution of a SDP is a policy sequence that maximizes the measure of the $\oplus$-sum of the rewards along all possible trajectories.

- Now we can express what it means for a policy sequence to be a solution of a SDP precisely:

$$\mathit{OptPolicySeq} : \{t\ n : \mathbb{N}\} \to \mathit{PolicySeq}\ t\ n \to \mathit{Set}$$
$$\mathit{OptPolicySeq}\ \{t\}\ \{n\}\ \mathit{ps} = \forall\, (\mathit{ps}' : \mathit{PolicySeq}\ t\ n) \to \mathit{val}\ \mathit{ps}' \leqslant_I \mathit{val}\ \mathit{ps}$$

Solution: optimality of policy sequences

- A solution of a SDP is a policy sequence that maximizes the measure of the $\oplus$-sum of the rewards along all possible trajectories.
- Now we can express what it means for a policy sequence to be a solution of a SDP precisely:

$$\begin{aligned} &\mathit{OptPolicySeq} \;:\; \{t\ n \;:\; \mathbb{N}\} \;\to\; \mathit{PolicySeq}\ t\ n \;\to\; \mathit{Set} \\ &\mathit{OptPolicySeq}\ \{t\}\ \{n\}\ \mathit{ps} \;=\; \forall\,(\mathit{ps}' \;:\; \mathit{PolicySeq}\ t\ n) \;\to\; \mathit{val}\ \mathit{ps}' \;\leqslant_l\; \mathit{val}\ \mathit{ps} \end{aligned}$$

Exercise 6.4

What is the type of $\leqslant_l$ in the definition of *OptPolicySeq*? Define $\leqslant_l$ in terms of $\leqslant$.